FinSight v2.0

Grid-Native Agentic Analysis for Structured Financial Documents

Technical Design & Engineering Documentation

"We adopted the shape that scales and kept the boundary that makes financial answers trustworthy."

Document Control

Field	Value
Document title	FinSight v2.0 — Technical Design & Engineering Documentation
Version	2.0
Status	Reference implementation — runnable end to end
Classification	Internal / Engineering
Primary artifact	Python package finsight (src/finsight), Streamlit UI, CLI
Audience	Engineering, solution architecture, technical leadership, evaluators
Runtime	Python 3.10+ (3.12 recommended)

Purpose. This document describes the problem FinSight solves, the scope and goals of the system, its architecture layer by layer, the engineering decisions behind it, and the operational concerns (configuration, security, reliability, evaluation) relevant to running it at an industrial standard. It is written to be read both top to bottom and as a reference.

Table of Contents

1. Executive Summary

FinSight is an analysis engine for structured financial documents — 10-K filings, annual reports, and the tables and prose inside them. It answers analytical questions that span many filings at once: comparisons of margins, ratios, trends, and qualitative disclosures across a set of companies.

Its defining idea is to treat a cross-document question as a grid: documents become rows, the sub-questions the question decomposes into become columns, and every intersection is an independently answered, independently audited cell. Cells run in parallel, each scoped to exactly one document, and each one traces back to a single source span.

The second defining idea is a hard trust boundary: every number leaves the language model and passes through a deterministic Python calculation engine the model cannot influence. Figures are extracted by deterministic parsers into a numeric store; ratios are computed by a fixed formula library; a missing input is refused, never guessed. The language model plans the grid and writes the narrative, but it never performs arithmetic. This is what makes the answers defensible for financial use.

The system runs fully offline with no API key (a deterministic mock backend exercises the entire pipeline), and upgrades transparently to a live LLM — Anthropic Claude or OpenAI — when a key is supplied. Heavy ML dependencies (BGE embeddings, FAISS, cross-encoder reranking) are optional and degrade gracefully to lightweight pure-Python fallbacks, so the reference implementation is demonstrable on any machine.

2. Problem Statement

2.1 Context

Financial analysts, investors, and corporate strategy teams routinely need to extract and compare specific figures and disclosures across many filings: revenue and net income across a peer set, leverage and liquidity ratios, multi-year growth trends, the risk factors each company discloses. The source material is long, semi-structured, and unforgiving — a single misread figure or a number attributed to the wrong company can invalidate an entire analysis.

2.2 The core problem

General-purpose retrieval-augmented generation (RAG) and single-loop agent systems are poorly suited to this task for three reasons:

- **Cross-contamination.** Blending many documents into one context lets one company's text leak into another company's answer. In finance this is not a cosmetic error; it is a wrong number.
- **Hallucinated arithmetic.** Language models are unreliable calculators. A net margin computed by an LLM may look plausible and be wrong, with no audit trail to catch it.
- **No provenance.** When an answer cannot be traced to a precise source span, it cannot be trusted, defended, or corrected.

2.3 Why a single blended loop falls short

A blended ReAct-style loop is excellent for a handful of documents but leaves performance and trust on the table when a question spans many filings. It cannot fan out cleanly across thousands of documents, it cannot guarantee isolation between documents, and it cannot guarantee that each answer maps to exactly one source span. FinSight v2.0 replaces that single loop with a planner plus many isolated per-document workers — the grid.

3. Goals and Non-Goals

3.1 Goals

- **Correct numbers first.** Figures and ratios must be exact and verifiable; numeric accuracy is the headline metric.
- **Isolation.** Each cell reads exactly one document; no cross-document contamination is possible by construction.
- **Auditability.** Every non-empty cell traces to one source span, with a route and a confidence.
- **Scale through parallelism.** Independent cells fan out across documents and sub-questions concurrently.
- **Graceful degradation.** The full pipeline runs offline with no key and no GPU; richer backends are optional.
- **Refusal over fabrication.** A missing figure is returned empty and flagged; a ratio depending on it is refused, not estimated.

3.2 Non-Goals

- Open-domain question answering or general web research.
- Replacing a language model's reasoning for qualitative synthesis — the LLM still writes the narrative.
- OCR of scanned/image-only PDFs (text must be machine-extractable).
- Real-time market data, pricing feeds, or trading signals.
- A production multi-tenant service; this is a single-user reference implementation with production-grade internals.

4. Scope

In scope	Out of scope
Ingesting txt, csv, pdf, docx, xlsx filings	Scanned/image PDFs requiring OCR
Cross-document numeric & ratio analysis	Live market / pricing data
Qualitative extraction (risks, outlook)	Investment recommendations / advice

In scope	Out of scope
Deterministic ratio computation & verification	LLM-performed arithmetic
Cited narrative synthesis	Multi-user auth, RBAC, billing
Offline mock + live LLM (Anthropic/OpenAI)	Fine-tuning or training models

5. Stakeholders and Use Cases

Stakeholder	Representative use case
Equity / credit analyst	Compare net margin, ROE, and leverage across a peer set of filings.
Corporate strategy	Summarize and contrast the risk factors and outlook each competitor discloses.
Solution architect / SE	Demonstrate a trustworthy, auditable document-analysis pattern.
Data / ML engineer	Extend retrieval, embeddings, or the formula library.
Technical evaluator	Inspect provenance per cell and validate numeric accuracy against ground truth.

6. Solution Overview

FinSight decomposes a question into a **documents × sub-questions** grid, answers every cell independently and in parallel (each scoped to a single document), verifies across cells, and synthesizes a cited narrative. The control-flow change from a single blended loop to a planner plus isolated per-cell workers is the headline of v2.0; the deterministic numeric core is carried over intact.

The end-to-end flow from a user query to a cited answer:

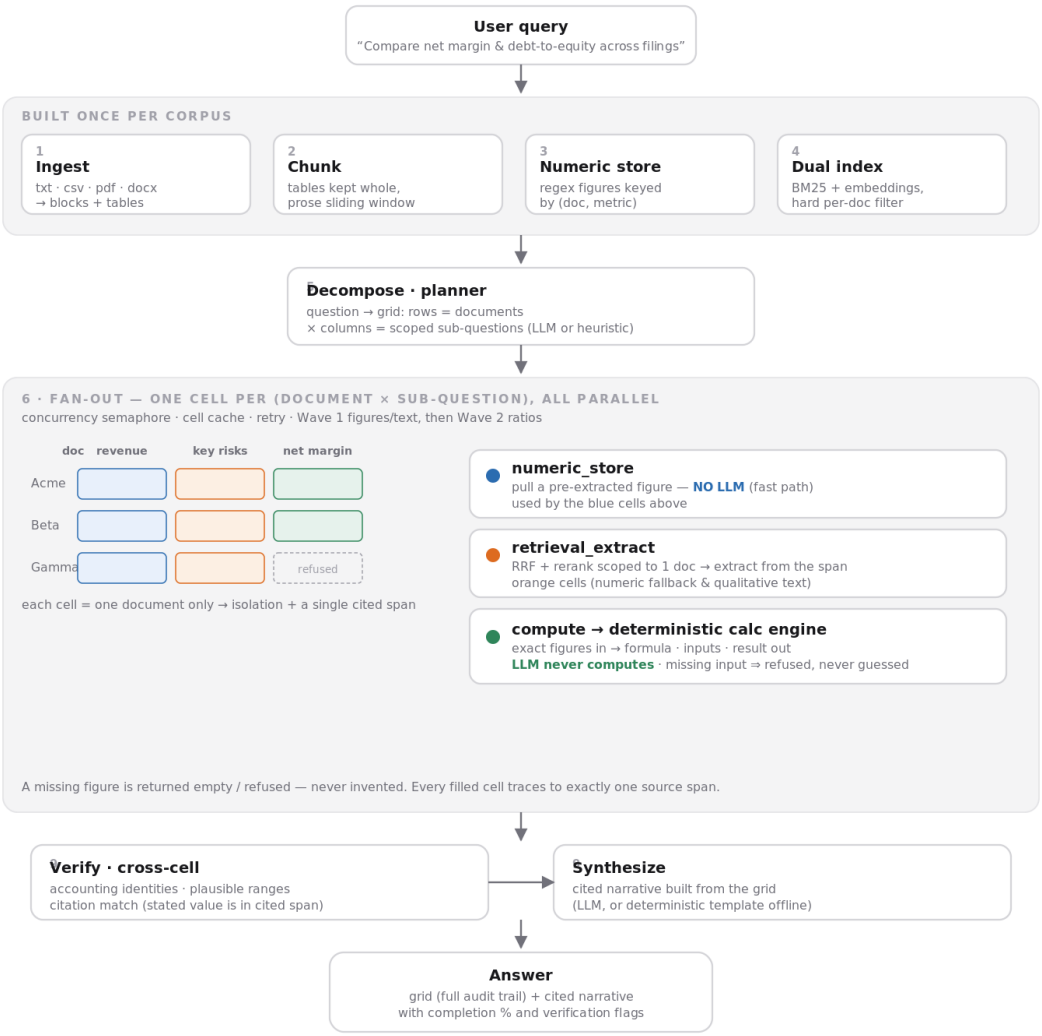

Figure 1. *Query-to-answer pipeline. The built-once front half (ingest, chunk, numeric store, dual index) is reused across queries; each query runs decompose → fan-out → verify → synthesize.*

7. Key Design Principles

1. **Determinism boundary.** Numbers never originate from the LLM. They are parsed deterministically and computed by a fixed engine. This is the system's load-bearing trust property.
2. **One cell, one document, one span.** Isolation and auditability are structural, not best-effort.
3. **Refuse, don't guess.** Missing inputs produce an explicit refusal rather than an invented figure.
4. **Pure-function cells.** A cell is a pure function of (document, sub-question), which is exactly why it caches cleanly and parallelizes safely.
5. **Graceful degradation.** Every heavy dependency has a lightweight fallback so the system always runs.

6. **Provider neutrality.** The LLM is an injectable, swappable component (Anthropic, OpenAI, or deterministic mock).

8. Domain Model

The grid data model is the atomic vocabulary of the system. Everything good about the grid — parallelism, isolation, caching, traceability — falls out of these structures (defined in src/finsight/models.py).

8.1 The Cell

A Cell is one (document, sub-question) unit of work and of audit.

Field	Meaning
doc_id	The single document this cell reads.
column_id / question	Which sub-question / field, and its scoped text.
status	pending · running · done · failed · empty.
value / unit / detail	The answer, its unit, and the compute/extract expression.
source	Exactly one Source span (file, page, section, char span).
confidence	0–1 trust score for the cell.
path	Which route filled it: numeric_store · retrieval_extract · compute.
error	Refusal or failure reason, when applicable.

8.2 Plan and Grid

- **ColumnSpec** — one sub-question: id, name, scoped question, type (numeric · text · ratio), and for ratios a formula plus the depends_on input columns.
- **GridPlan** — the planner's strict structured output: the question, the columns, and the rows (documents in scope).
- **Grid** — rows × columns of Cells, plus the narrative and verification notes.
- **Source** — file, page, section, and character span: the single provenance record per cell.

9. System Architecture — Layer by Layer

The pipeline (orchestrated in pipeline.py) is built from ten layers plus two cross-cutting concerns. The front half (layers 1–4) is built once per corpus and reused across queries; layers 5–10 run per query.

#	Layer	Role
1	Format-aware ingestion	Parse txt/csv/pdf/docx/xlsx into a unified block + table schema.

#	Layer	Role
2	Chunking	Tables kept whole; prose sliding-window; metadata (incl. doc_id) on every chunk.
3	Numeric store	Regex extraction of figures keyed by (doc, metric) — the cell fast path.
4	Dual indexing	BM25 + embeddings with a hard per-document filter.
5	Decompose	Plan the grid: rows = documents, columns = sub-questions.
6	Cell engine (fan-out)	Execute every cell in parallel via one of three routes.
7	Scoped retrieval	RRF fusion + optional rerank, hard-filtered to one document.
8	Calc engine	Deterministic formula library; the trust boundary.
9	Verify & synthesize	Cross-cell checks, then a cited narrative.
10	Frontend	CLI and Streamlit grid UI with per-cell provenance.

9.1 Layer 1 — Format-Aware Ingestion

Files are routed by type and parsed into a unified schema of Blocks (a span of prose or a whole table) carrying page, section, and character-span metadata. Text/CSV parse with zero native dependencies; PDF (pdfplumber/PyMuPDF), DOCX (python-docx), and XLSX (openpyxl) are wired behind the optional [ingest] extra. A stable doc_id is derived from a hash of the filename so evaluation ground truth can reference documents reproducibly across machines.

9.2 Layer 2 — Chunking

Tables are never split — they are kept whole so a figure is never severed from its row label. Prose is chunked with a sliding window that prefers sentence/whitespace boundaries. Every chunk carries its doc_id, which the per-document retrieval filter depends on.

9.3 Layer 3 — Structured Extraction → Numeric Store

A regex pre-filter scans tables and prose for known metric phrases (a controlled vocabulary mapping surface phrases like "total revenue" to canonical keys like revenue), parses the adjacent figure (applying magnitude suffixes such as "million"/"billion" and treating parenthesized values as negatives), and records a validated NumericFact keyed by (doc_id, metric) with a confidence score. Table-sourced facts outrank prose-sourced facts. This store doubles as the cell fast path: a numeric cell answerable from it skips the LLM entirely.

9.4 Layer 4 — Dual Indexing + Per-Document Filter

A dense index (BGE embeddings via sentence-transformers, or a deterministic hashed bag-of-words fallback in pure NumPy) and a sparse BM25 index are built over all chunks. The single most important

property is a hard per-document filter applied before scoring: there is no API to search across documents, which is what structurally guarantees cell isolation.

9.5 Layer 5 — Query Decomposition & Grid Construction

The planner reads the question and the document set and emits a GridPlan as strict structured output. With a live LLM it decomposes flexibly; offline, a deterministic heuristic maps financial phrasing to numeric, ratio, and text columns and — critically — auto-adds the input columns a ratio depends on (you cannot compute net margin without net-income and revenue columns present). A repair pass guarantees every ratio's dependency columns exist even if the LLM omits them. This is the highest-leverage step; the quality of the whole system rides on it.

9.6 Layer 6 — Per-Cell Execution Engine (Fan-Out)

The grid is filled in two dependency-respecting waves — figures/text first, then the ratios that consume them — each wave fully parallel internally. Concurrency is bounded by a semaphore (the cost-control valve), each cell checks the cache first, and transient failures retry with exponential backoff. Every cell takes exactly one of three routes (Section 10).

9.7 Layer 7 — Scoped Retrieval & Reranking

Invoked per cell with a mandatory document filter. Dense and sparse hits are fused by Reciprocal Rank Fusion (RRF); an optional cross-encoder reranks the top candidates, otherwise the fused order stands. This is a subroutine of the cell engine, never a single blended retrieval over mixed documents.

9.8 Layer 8 — Deterministic Calculation Engine

The trust boundary. Exact figures in; formula name, inputs, human-readable expression, and result out. Unsupported formulas and missing or non-numeric inputs are refused via a typed error, never approximated. Division-by-zero and boolean inputs are rejected explicitly. The LLM never reaches this code.

9.9 Layer 9 — Cross-Cell Verification & Synthesis

Three checks run across the filled grid: row identities (e.g., gross profit must not exceed revenue; equity must not exceed total assets), column consistency (margins outside a plausible band are flagged), and citation match (a stated figure must actually appear in its cited span, magnitude-aware). The verified grid is then synthesized into a cited narrative — by the LLM when live, or a deterministic template offline.

9.10 Layer 10 — Frontend

Two entry points share the same pipeline: a colorized CLI (finsight info | ask | eval) and a Streamlit grid UI. The UI supports drag-and-drop upload or a folder path, streams the grid, color-codes each cell by route, and exposes per-cell click-through provenance (value, confidence, route, source span).

9.11 Cross-Cutting — Caching and Evaluation

A cell-level disk cache and an evaluation harness sit across the pipeline; both are detailed in Sections 13 and 15.

10. The Three Cell Routes

Every cell resolves through exactly one route. Two of the three never invoke the LLM — which is why numeric and ratio questions are exact and cheap.

Route	How it answers	LLM?
numeric_store	Pull a pre-extracted figure from the store (fast path).	No
retrieval_extract	Scoped RRF + rerank in one document, then extract the figure or qualitative answer from the span.	Text cells: yes · Numeric fallback: no
compute	Call the deterministic calc engine with exact figures from peer cells.	No

A numeric cell tries the store first; on a miss it falls back to scoped retrieval-and-scan; if still not found it returns empty and flagged. A ratio cell reads its already-filled input cells and calls the calc engine; if any input is missing it is refused. A text cell retrieves the top span and asks the LLM to answer strictly from that span (or returns empty).

11. Deterministic Calculation Engine

The formula library (calc/engine.py) is a set of pure functions over named numeric inputs. Each returns a full audit trail: the formula name, the exact inputs used, a rendered expression, and the result.

Formula	Inputs	Output
net_profit_margin	net_income, revenue	%
gross_margin	gross_profit, revenue	%
operating_margin	operating_income, revenue	%
return_on_equity	net_income, total_equity	%
return_on_assets	net_income, total_assets	%
current_ratio	current_assets, current_liabilities	x
debt_to_equity	total_debt, total_equity	x
yoy_growth	current, prior	%
cagr	begin, end, years	%

Formula	Inputs	Output
eps	net_income, shares_outstanding	$

Refusals are a feature, not a bug. An unsupported formula, a missing input, a non-numeric input, or a zero denominator all raise a typed CalcError that surfaces as an explicit refusal on the cell — a missing figure is never computed on a guess.

12. LLM Integration Layer

The LLM is an injectable component (llm/client.py) with a small, uniform contract: structured output (a validated Pydantic object) and free-form completion, each at a routable model tier ("small" or "large").

12.1 Providers

- **Anthropic** — structured output via forced single-tool use; tiers map to configured Claude models.
- **OpenAI** — structured output via a forced function call; tiers map to configured GPT models (e.g., gpt-4o-mini).
- **Mock (offline)** — a deterministic backend. The data-owning layer precomputes a best-effort answer and passes it as a hint; the mock validates it back into the requested schema. Intelligence lives in the layer that owns the data, never hidden in the mock.

12.2 Model routing and selection

Callers request a tier; the client resolves it to a concrete model per active provider. Provider, keys, and model names are all configuration. TLS for live calls is pinned to certifi's CA bundle so connectivity does not depend on the host's system certificate store (a common failure mode on fresh macOS Python builds).

13. Cost Model — the D × C Tax and Mitigations

A grid issues up to D documents × C columns model calls. Four mitigations make the common case fast and cheap, and all are implemented:

- **Concurrency cap.** A semaphore bounds in-flight calls (FINSIGHT_MAX_CONCURRENCY), protecting rate limits.
- **Cell-level cache.** A cell is a pure function of (document, sub-question); re-running a grid after one column changes recomputes only that column.
- **Model routing.** Simple extraction cells use a small model; only hard cells, planning, and synthesis need a frontier model.
- **Numeric fast path.** Cells answerable from the numeric store skip the LLM entirely.

14. Caching Strategy

The cell cache (diskcache-backed) keys each entry on a content signature of (doc_id, question, type, formula, depends_on). Editing one column's question invalidates only that column's cells while every other cell stays a hit. Only terminal states (done, empty) are cached; transient failures are never cached.

Known caveat. The signature includes the document id but not the document's content. Because the doc_id derives from the filename, re-uploading a same-named file with changed numbers can serve stale cells. The UI upload path mitigates this by namespacing the cache to a hash of the uploaded content; for the folder path, clear the cache after editing a file in place.

15. Reliability and Error Handling

- **Retry with backoff.** Transient cell failures and retryable API errors (HTTP 429 and 5xx, timeouts, connection errors) retry with exponential backoff; 4xx client errors are not retried.
- **Refusal semantics.** Missing or invalid inputs yield an explicit, recorded refusal rather than a fabricated value.
- **Isolation of failure.** A failed cell is recorded and contained; it does not corrupt peers or halt the grid.
- **TLS pinning.** Live clients verify against certifi, decoupling connectivity from host SSL configuration.
- **Offline guarantee.** With no key, the full pipeline still executes deterministically end to end.

16. Evaluation Methodology

The grid makes three metrics natural, and adds the one a financial tool most needs — is the number actually right — which none of the standard RAGAS retrieval metrics capture.

Metric	Definition
Cell Numeric Accuracy	Fraction of numeric cells equal to ground truth (within tolerance). The headline number.
Grid Completion	Fraction of cells filled rather than failed/empty.
Attribution Correctness	Whether each cell's cited span actually contains its stated value (magnitude-aware).
RAGAS (optional)	Faithfulness / relevancy / precision / recall, behind the [eval] extra.

On the bundled three-filing demo with ground truth, the reference configuration reports 100% cell numeric accuracy and 100% attribution correctness. Note that low grid completion on open-ended questions is expected and healthy: an ambitious planner may create columns the filings do not disclose, which correctly resolve to refused/empty rather than fabricated values.

17. Configuration and Deployment

17.1 Key settings (env, FINSIGHT_ prefix)

Setting	Purpose	Default
LLM_PROVIDER	anthropic | openai	anthropic
ANTHROPIC_API_KEY / OPENAI_API_KEY	Provider credential (blank ⇒ mock)	(blank)
MODEL_SMALL / MODEL_LARGE	Anthropic tier models	Haiku / Opus
OPENAI_MODEL_SMALL / _LARGE	OpenAI tier models	gpt-4o-mini
MAX_CONCURRENCY	Fan-out semaphore size	8
MOCK_LLM	Force offline mock even with a key	false

17.2 Optional dependency extras

Extra	Adds
[ml]	BGE embeddings (sentence-transformers), FAISS, cross-encoder rerank.
[ingest]	Real PDF/DOCX/XLSX parsing (pdfplumber, PyMuPDF, python-docx, openpyxl).
[ui]	Streamlit grid UI (streamlit, streamlit-aggrid).
[eval]	RAGAS retrieval metrics.
[dev]	pytest, pytest-asyncio, ruff.

17.3 Quickstart

7. Create a venv and install: pip install -e ".[ui]" (add [ingest] for real PDFs).
8. Run offline: finsight info / finsight ask "..." / finsight eval — no key required.
9. Go live: copy .env.example to .env, set the provider and key, restart.
10. UI: streamlit run app/streamlit_app.py, upload filings, ask a cross-document question.

18. Security and Compliance Considerations

- **Secret handling.** Keys live in a gitignored .env and are never committed; the client trims and validates them. Keys pasted into shared channels should be rotated.
- **Data residency.** In mock mode no data leaves the machine. In live mode, only scoped spans and the grid table are sent to the chosen provider; raw documents are not bulk-uploaded.
- **Least exposure.** The numeric core, calc engine, and verification run locally regardless of provider.

- **Auditability.** Per-cell provenance supports review and post-hoc verification of every figure.
- **Determinism for sensitive use.** Because numbers never depend on the model, switching providers or running offline does not change the figures.

19. Performance and Scalability

- **Parallel fan-out.** Independent cells run concurrently under a semaphore; throughput scales with the concurrency cap and provider limits.
- **Built-once front half.** Ingestion, chunking, the numeric store, and the index are constructed once per corpus and reused across queries.
- **Cache amortization.** Repeated (document, sub-question) pairs are free; iterating on one column recomputes only that column.
- **Cheap fast path.** Numeric and ratio cells avoid the LLM entirely, so figure-heavy questions are dominated by local compute.
- **Backend scaling.** Embeddings/index swap from pure-Python fallbacks to BGE + FAISS without code changes for larger corpora.

20. Technology Stack

Concern	Reference (default)	Heavy (optional)
Language / runtime	Python 3.10+ (asyncio)	—
Data model / validation	Pydantic v2, pydantic-settings	—
Dense embeddings	Hashed bag-of-words (NumPy)	BGE (sentence-transformers)
ANN index	Brute-force NumPy cosine	FAISS
Sparse retrieval	rank-bm25	rank-bm25
Reranking	RRF order	Cross-encoder
LLM	Deterministic mock	Claude / OpenAI
Calc / verify	Pure Python	—
Persistence	diskcache, DuckDB, pandas	—
Frontend	CLI; Streamlit + AgGrid	—

21. Testing Strategy

The suite (pytest, async-aware) covers extraction, indexing, the calc engine, the eval metrics, and an end-to-end pipeline integration test, all runnable offline in well under a second. Determinism makes the tests stable: with the mock backend the pipeline produces the same grid every run, so regressions

surface immediately. Numeric correctness is additionally validated against bundled ground truth via finsight eval.

22. Limitations and Known Issues

- Extraction is regex/vocabulary-driven: figures must read like statements; image-only PDFs and unconventional layouts may yield empty cells.
- Table magnitude is read per-cell; a bare "18000" without a "million" suffix is taken literally unless the header magnitude is embedded.
- The offline planner is keyword-based: questions outside its catalog fall back to a single text column.
- Live planners (especially smaller models) can over-generate unanswerable columns, lowering completion (figures remain correct).
- The cell cache keys on filename, not content (see Section 14).
- Single-user reference scope: no auth, quotas, or multi-tenant isolation.

23. Future Roadmap

- Planner guardrails to suppress unanswerable columns on open-ended questions.
- Content-hash doc ids to fully close the stale-cache caveat on the folder path.
- Period-aware extraction (multi-year columns) and header-magnitude propagation for tables.
- Live cell streaming in the UI and one-click grid/CSV export.
- Expanded formula library and configurable verification identities.
- Optional vector-DB backend for very large corpora.

Appendix A — Module Map

Path	Responsibility
models.py	Grid data model: Cell, Grid, GridPlan, ColumnSpec, Source.
config.py	Settings, provider selection, optional-dependency capability flags.
pipeline.py	Orchestrates all ten layers.
ingestion/ingest.py	Layer 1 — format-aware parsing.
chunking/chunker.py	Layer 2 — chunking.
extraction/numeric_store.py	Layer 3 — numeric store + scan.
indexing/dual_index.py, embeddings.py	Layer 4 — dual index + per-doc filter.

Path	Responsibility
grid/decompose.py	Layer 5 — planner.
grid/cell_engine.py	Layer 6 — fan-out execution + routes.
retrieval/retriever.py	Layer 7 — RRF + rerank.
calc/engine.py	Layer 8 — deterministic formulas.
grid/verify.py	Layer 9 — verification + synthesis.
grid/store.py	Grid → DuckDB / CSV persistence.
cache/cell_cache.py	Cell-level cache.
eval/metrics.py	Evaluation metrics.
llm/client.py, mock.py	LLM client (Anthropic/OpenAI) + offline mock.
cli.py, app/streamlit_app.py	Layer 10 — CLI and grid UI.

Appendix B — Canonical Metric Vocabulary (excerpt)

The extractor maps surface phrases to canonical keys (longest-phrase-first to avoid partial matches). A representative subset:

- revenue ← total revenue, net revenue, net sales, total sales, revenue, sales
- net_income ← net income, net earnings, net profit, profit for the year
- gross_profit ← gross profit; operating_income ← operating income, income from operations
- total_assets, total_liabilities, total_equity (shareholders'/stockholders' equity)
- current_assets / current_liabilities (total current …)
- total_debt (incl. long-term debt), cash (cash and cash equivalents), shares_outstanding, rd_expense

Appendix C — Glossary

Term	Meaning
Cell	One (document, sub-question) unit of work and audit.
Grid	Documents (rows) × sub-questions (columns) of cells.
Route	How a cell is filled: numeric_store, retrieval_extract, or compute.
Numeric store	Pre-extracted figures keyed by (doc, metric); the LLM-free fast path.
RRF	Reciprocal Rank Fusion — combines dense and sparse rankings.
Determinism boundary	The rule that all numbers come from code, never the LLM.
Refusal	Explicit non-answer when an input is missing/invalid — never a guess.

Term	Meaning
D × C tax	Up to documents × columns model calls; controlled by four mitigations.

End of document.